

Subprograme (funcții) în Python

Fișă de lucru — Informatică, clasa a IX-a (matematică-informatică, regim intensiv)

Nivel de vârstă și utilitate

Clasa a IX-a, filiera teoretică, profil real, specializarea matematică-informatică, regim intensiv de predare a informaticii.

Materialul introduce conceptul de subprogram (funcție) ca bloc de cod reutilizabil, esențial pentru organizarea clară a programelor Python. Este potrivit ca prim contact formal cu funcțiile definite de utilizator, după ce elevii cunosc deja instrucțiunile de bază din gimnaziu.

1. Noțiuni teoretice

Un subprogram (funcție) este un bloc de instrucțiuni cu un nume, care poate primi date de intrare (parametri) și poate returna un rezultat. Analogie: o rețetă de bucătărie primește ingrediente (parametri) și produce un preparat (rezultatul returnat).

- Definirea unei funcții se face cu cuvântul cheie `def`, urmat de nume și parametri, apoi corpul funcției (indentat).
- Rezultatul se transmite înapoi cu instrucțiunea `return`; o funcție fără `return` (sau cu `return` fără valoare) întoarce implicit `None`.
- O funcție poate avea oricâți parametri: zero, unul sau mai mulți — numărul lor depinde exclusiv de ce date are nevoie funcția, nu există o regulă de „doi parametri”.
- Apelul unei funcții nu înseamnă neapărat o atribuire. Rezultatul poate fi reținut într-o variabilă (`x = functie(...)`), poate fi folosit direct într-o expresie (`print(funcție(...))`) sau funcția poate fi apelată ca instrucțiune de sine stătătoare, atunci când efectul ei este, de exemplu, o afișare pe ecran, nu obținerea unei valori.
- Variabilele definite în interiorul unei funcții sunt locale — există doar cât timp se execută funcția.
- Python oferă funcții predefinite utile: `abs()`, `round()`, `int()` pentru calcule numerice, respectiv `len()`, `min()`, `max()`, `sum()` pentru colecții (liste).

Sintaxă generală — numărul de parametri variază

```
def fara_parametri():  
    # instrucțiuni, fără date primite din exterior  
    ...  
  
def cu_un_parametru(x):  
    # instrucțiuni care folosesc x  
    ...  
  
def cu_mai_multi_parametri(p1, p2, ..., pn):  
    # n poate fi orice număr: 2, 3, 4, 5...  
    rezultat = ...  
    return rezultat
```

2. Exemple rezolvate

Exemplul 1 — funcție cu doi parametri, care calculează media aritmetică a două numere (apel prin atribuire):

```
def media(a, b):
    """Returnează media aritmetică a numerelor a și b."""
    return (a + b) / 2

rezultat = media(7, 9)  # apel prin atribuire
print(rezultat)         # afișează 8.0
```

Exemplul 2 — funcție cu un singur parametru, apelată direct într-o expresie, fără atribuire intermediară:

```
def dublu(x):
    return x * 2

print(dublu(5))  # apel folosit direct în print(), fără variabilă intermediară
```

Exemplul 3 — funcție care NU returnează nimic (nu are return); apelul se face ca instrucțiune de sine stătătoare:

```
def afiseaza_salut(ume):
    print("Salut,", ume + "!")
    # nu există return -> funcția întoarce None

afiseaza_salut("Ana")  # apel ca instrucțiune; NU se face x = afiseaza_salut("Ana")
```

Exemplul 4 — funcție fără niciun parametru, care folosește o funcție predefinită pentru colecții:

```
def medie_note():
    note = [8, 9, 10, 7]  # datele sunt fixate în interiorul funcției
    return sum(note) / len(note)

print(medie_note())  # afișează 8.5
```

3. Exerciții propuse

1. Scrieți o funcție `maxim(a, b)` care returnează valoarea mai mare dintre `a` și `b`, fără a folosi funcția predefinită `max()`.
2. Scrieți o funcție `este_par(n)` care returnează `True` dacă numărul întreg `n` este par și `False` altfel.
3. Folosind modulul `math`, scrieți o funcție `distanța(x1, y1, x2, y2)` care calculează distanța euclidiană dintre punctele `(x1, y1)` și `(x2, y2)`, utilizând `math.sqrt()`.

4. Scrieți un program care citește o listă de note ale unui elev și, folosind funcțiile predefinite `len()`, `min()`, `max()` și `sum()`, afișează numărul de note, nota minimă, nota maximă și media.

4. Barem de corectare

Cerință	Punctaj
Exercițiul 1 — definire corectă (0,5p) + testare cu cel puțin două perechi de valori (0,5p)	1p
Exercițiul 2 — condiție de paritate corectă (0,5p) + testare (0,5p)	1p
Exercițiul 3 — utilizare corectă a <code>math.sqrt()</code> (1p) + testare (0,5p)	1,5p
Exercițiul 4 — utilizare corectă a celor 4 funcții predefinite (1,5p) + afișare clară a rezultatelor (0,5p)	2p
Claritate, comentarii relevante și denumiri sugestive ale variabilelor/funcțiilor	1p
Total	10p (echivalat: 1p din examenul scris)

5. Bibliografie și webografie

- Python Software Foundation — „Defining Functions”, docs.python.org/3/tutorial/controlflow.html#defining-functions
- w3schools.com — „Python Functions”, w3schools.com/python/python_functions.asp
- Programa școlară Informatică, clasa a IX-a, curriculum de specialitate, Anexa nr. 49, OMEC 2025–2026.